

《程序设计基础》大作业报告

Qbsidian：基于 Qt 的知识图谱与 Markdown 笔记软件

组号：70

一、程序功能介绍

1.1 项目背景

传统笔记软件多采用树状目录进行线性存储，难以体现知识点之间的关联关系。本项目基于 C++17 与 Qt Widgets 框架，开发一款轻量级本地知识管理工具 **Qbsidian**，借鉴 Obsidian 的核心理念，重点实现**双向链接**与**可视化知识图谱**，将离散的 Markdown 笔记交织成知识网络，并提供检索、复习规划、题目练习与 AI 助教等辅助功能。

1.2 核心功能

(1) 双栏 Markdown 编辑与文件管理

- 本地知识库**：启动时选择本地文件夹作为知识库，左侧树状视图展示多层次目录结构。
- 双栏实时渲染**：左侧纯文本编辑，右侧实时 HTML 预览，支持标题、列表、加粗、斜体、行内代码、代码块、引用块、分隔线等基础 Markdown 语法。
- 文件操作**：支持新建笔记/文件夹、重命名、删除、保存、另存为，所有笔记以 UTF-8 纯 .md 文件持久化存储。

(2) 双向链接机制

- 通过正则表达式识别 `[[笔记名]]` 与 `[[笔记名|别名]]` 语法。
- 预览区将其渲染为内部超链接，点击后自动定位或创建对应笔记，实现知识网络中的无缝跳转。

(3) 知识库检索

- 提供全局搜索侧边栏，遍历知识库下所有文本文件，匹配文件名、正文内容与 `#标签`。
- 结果展示文件名与上下文摘要，点击即可打开对应文件。

(4) 动态知识图谱

- 后台扫描所有 .md 文件，提取双向链接关系构建图结构。
- 采用**力导向图算法**，模拟节点排斥力、边弹簧引力、同目录弱连接与中心重力，动态渲染为可交互的知识网络星空图，支持拖拽、悬停、双击打开笔记。

(5) 复习规划器

- 为每篇笔记维护学习状态（创建时间、最近复习时间、下次复习时间、复习次数、策略等）。
- 内置艾宾浩斯策略（1/2/4/7/15/30 天间隔）与自定义策略（用户指定间隔并规避休息日）。
- 启动时自动生成“今日待办”，支持手动排期与拖拽调整。

(6) 题目抽查练习系统

- 通过自定义语法 `#题库/题目名 问题::答案` 从笔记中抽取题目。

- 弹出独立练习窗口，正面显示问题，查看答案后标记“认识/不认识”，反馈影响后续复习权重。

(7) AI 助教

- 接入 DeepSeek API，支持自由对话。
- 提供“一键笔记总结”与“一键生成题库”快捷功能。

二、项目各模块与类设计细节

2.1 总体架构

项目采用 Qt Widgets 桌面应用架构，CMake 构建，源码集中在 `Qbsidian/` 目录，按职责划分为中心层、UI 层、功能控制层、文档解析层与工具辅助层。

模块层	主要文件	职责
中心层	<code>main.cpp</code> , <code>mainwindow.h/cpp/ui</code>	程序入口、主窗口
UI 层	<code>fileexplorerpane</code> 等 8 个 Pane	文件树、编辑预览、复习时间轴、AI 对话、练习弹窗、知识图谱
功能控制层	<code>notemanager</code> 等 6 个类	文件 I/O、复习排期、题目抽取、AI 网络请求
文档解析层	<code>markdownparser</code> 等 4 个类	Markdown 分块、行内语法、HTML 渲染
工具辅助层	<code>autohidescrollareafilter</code> 等	滚动条自动隐藏、行号区域

2.2 核心类设计

Mainwindow（主控制器）

`Mainwindow` 采用组合方式持有所有子系统，负责装配 UI 与业务对象、处理信号槽总线以及管理标签页生命周期。

- 内部结构 `NoteTab`：封装单个标签页内的 `EditorPane`、`PreviewPane`、`QSplitter` 与文件路径。
- 核心成员：`m_fileExplorer`、`m_tabwidget`、`m_reviewManager`、`m_noteManager`、`m_aiAgent`、`m_graphPane`、`QHash<QString, NoteTab*> m_tabsByPath`。
- 核心方法：`connectSignals()` 集中连接信号槽；`openFileInTab()`、`saveTab()` 管理多标签状态；`onAnchorClicked()` 处理内部链接与外部 URL。

NoteManager（本地知识库）

负责文件系统操作与全局检索。

- 核心方法：`load()`、`save()`、`createNewNote()`、`createNewFolder()`、`searchInVault()`、`findNotePath()`。

EditorPane / PreviewPane（双栏编辑预览）

- `EditorPane` 继承 `QPlainTextEdit`。
- `PreviewPane` 继承 `QTextBrowser`，`showHtml()` 调用 `MarkdownParser::parse()` 并附加题库卡片样式。

MarkdownParser / BlockParser / RenderEngine / InlineParser (Markdown 渲染)

- `MarkdownParser`：静态入口，先归一化换行，再调用 `BlockParser::parseBlocks()` 与 `RenderEngine::render()`。
- `BlockParser`：逐行扫描生成 `LogicalBlock` 数组，识别标题、列表、代码块、引用、分隔线。
- `RenderEngine`：将 `LogicalBlock` 转换为 HTML；对列表递归构建 `ListNode` 树以支持嵌套。
- `InlineParser`：处理加粗、斜体、删除线、行内代码、图片、超链接、双向链接 `[[...]]`。

ReviewManager / ReviewEntity / IReviewStrategy (复习系统)

- `ReviewManager`：维护 `QMap<QString, ReviewEntity> m_noteEntities`；提供 `generateDailyPlan()`、`processReviewFeedback()`、`reviewPlanBetween()` 等接口；通过 `loadData()` / `saveData()` 读写 `{vault}/.qbsidian/review_data.json`。
- `ReviewEntity`：复习状态实体，包含 `noteId`、`strategyId`、`status`、`createTime`、`lastReviewTime`、`nextReviewTime`、`reviewCount`、`excludedReviewDates`，并提供 `toJson()` / `fromJson()`。
- `IReviewStrategy`：策略模式接口；`EbbinghausStrategy` 实现固定间隔；`CustomStrategy` 支持自定义间隔与休息日规避。

ForceDirectedGraph / GraphPane / GraphNodeItem (知识图谱)

- `ForceDirectedGraph`：纯物理模型，维护节点表、边列表与隐藏边列表；参数包括 `m_repulsionK`、`m_springK`、`m_idealLength`、`m_damping`、`m_gravityK`。
- `GraphPane`：基于 `QGraphicsScene/QGraphicsView` 的视图层。
- `GraphNodeItem`：继承 `QGraphicsObject`，负责节点绘制、悬停效果与双击交互。

AIAgent (AI 助教)

使用 `QNetworkAccessManager` 向 DeepSeek API 发送请求，封装 `askQuestion()`、`summarizeNote()`、`generateQuiz()`、`sendPostRequest()`。

QuestionExtractor (题库抽取)

通过正则 `#题库/(\\s+)\\s+(.+?)::(.+)` 扫描 `.md` 文件，生成 `ExtractedQuestion` 列表。题目 ID 使用 MD5（路径 + 行号）生成，保证复习进度稳定可追踪。

2.3 类间协作关系

- **主从持有**：`Mainwindow` 组合 `NoteManager`、`ReviewManager`、`AIAgent`、各 `Pane`。
- **信号槽总线**：`Mainwindow::connectSignals()` 集中连接 `FileExplorerPane` 的导航、搜索、复习、图谱、AI 请求；`AIAgent::responseReceived` 回到 `Mainwindow` 更新 `AIAssistantPane`。
- **编辑预览同步**：标签页内 `EditorPane` 文本变更 → `QTimer` → `PreviewPane::showHtml()`，形成实时双栏同步。

- **双向链接跳转**: `InlineParser` 将 `[[笔记名]]` 渲染为 `<a href="internal:?note=...">`; `PreviewPane` 拦截 `anchorClicked` 信号, `MainWindow::onAnchorClicked()` 调用 `NoteManager::findNotePath()` 定位并打开。
- **复习数据流**: `ReviewTimelinePane` 展示 `ReviewManager::reviewPlanBetween()` 返回的计划; 用户反馈经 `processReviewFeedback()` 更新 `ReviewEntity` 并重新规划复习计划。
- **图谱构建**: `GraphPane::rebuildGraphData()` 遍历知识库提取双向链接, 构造 `ForceDirectedGraph` 节点与边; 节点双击通过 `noteOpenRequested` 信号回到 `MainWindow` 打开笔记。

2.4 核心数据结构与算法

力导向图物理模型

`ForceDirectedGraph::updatePhysicsStep()` 每帧执行:

1. 初始化节点合力为零。
2. 两两计算库仑排斥力: $F_{\text{repel}} = k_{\text{repel}} / d^2$ 。
3. 对每条边计算弹簧引力: $F_{\text{spring}} = k_{\text{spring}} \cdot (d - L_{\text{ideal}})$ 。
4. 同目录笔记间添加隐藏边, 以较弱引力促进聚类。
5. 施加中心重力使整体保持居中。
6. 速度更新: $v = (v + F) \cdot \text{damping}$, 再更新位置。

复习排期算法

- `ReviewEntity` 记录下次复习时间; `generateDailyPlan()` 过滤 `nextReviewTime <= now` 且状态为 `Learning` 的实体。
- 排序规则: 先按 `userPriority` 降序, 再按 `nextReviewTime` 升序, 最后按 `maxDailyLimit` 截断。
- `reviewPlanBetween()` 对未来日期区间做策略推演, 生成时间轴预览。

Markdown 解析流程

1. **Block 阶段**: 逐行识别块级语法, 输出 `QVector<LogicalBlock>`。
2. **Render 阶段**: 遍历块序列, 对列表递归建 `ListNode` 树; 段落、标题等调用 `InlineParser::process()`。
3. **Post-process 阶段**: `PreviewPane` 用正则把 `#题库/... :: ...` 替换为可折叠题库卡片 HTML。

2.5 设计亮点

- **力导向图物理引擎与聚类优化**: `ForceDirectedGraph` 纯物理模型独立计算库仑排斥力、弹簧引力与中心重力, 实现节点自然分布。为进一步提升可视化美观度, 我们在同级目录的笔记节点间引入**隐藏边弱引力**, 使知识结构在力导向布局中呈现自然的目录聚类效果, 兼顾全局拓扑与局部组织。
- **复习规划的多样性**: 通过 `IReviewStrategy` 策略模式抽象复习算法, 内置艾宾浩斯遗忘曲线策略 (1/2/4/7/15/30 天间隔) 与自定义策略 (用户可自由指定间隔并规避休息日); 同时支持 `userPriority` 优先级排序与 `maxDailyLimit` 每日上限截断, 实现多维度、可扩展的个性化复习排期。
- **AI 助教快捷功能**: `AIAgent` 封装 DeepSeek API 调用, 除自由对话外, 提供**"一键笔记总结"**与**"一键生成题库"**快捷入口, 将 AI 能力无缝嵌入笔记工作流, 降低用户操作成本。
- **Markdown 解析流水线设计**: 渲染层采用"先切块分类、再解析逐行"的两阶段架构——`BlockParser` 先将全文按空行切分为逻辑块并识别块级类型 (标题、列表、代码块、引用

等)，再由 `RenderEngine` 遍历各块调用 `InlineParser` 处理行内语法。该设计使块级结构与行内样式解耦，便于独立扩展新语法。

- **组合优于继承**：`Mainwindow` 通过成员对象组合各 `Pane`，`NoteTab` 结构体封装标签页状态，避免深继承层次带来的耦合。
- **模型-视图分离**：`ForceDirectedGraph` 纯物理模型不依赖 Qt 视图；`GraphPane` 负责视图同步，符合 MVC 思想。
- **稳定题目 ID**：使用 MD5（路径 + 行号）生成题目 ID，保证复习进度在文件重命名后仍可稳定追踪。
- **UI 主题一致性**：`Mainwindow` 统一下发 QSS，并调用各 `Pane` 的 `setDarkMode()` 同步切换，保证暗色/亮色模式全局一致。

三、小组成员分工情况

姓名	学号	主要分工	完成内容
李昊轩	2500012942	底层算法构建	负责底层算法：知识图谱物理引擎、搜索模块、复习规划等
孟祥祯	2500011022	ui界面完成	负责 UI 设计与实现：界面布局、交互设计、视觉优化等

整体的各个功能的构思和设计、以及完成后的优化是共同讨论打磨的成果,两人在开发过程中保持密切沟通

四、项目总结与反思

4.1 主要收获

通过本次大作业，小组系统实践了 C++ 面向对象程序设计的核心思想，并在多个维度获得了一些体会思考：

- **面向对象的问题拆解能力**：深入理解了"功能与属性各自独立"的封装思想，学会将复杂需求拆解为一个个职责清晰的类，明确每个类的属性（数据成员）与行为（成员函数），并通过接口定义模块间的交互，显著提升了代码的可维护性。
- **数据结构与算法实践**：实现了力导向图物理模拟（库仑力、弹簧力、阻尼衰减）、复习日期规划（策略模式 + 优先级排序）、正则解析等算法，体会到算法和数据结构在一个真实项目中的作用。
- **GitHub 协作与分工**：通过 Git 分支管理进行协作，体会到版本控制在多人并行开发中的必要性，也锻炼了沟通与冲突解决能力。
- **模块化与接口预留意识**：认识到在制定项目规划时，应提前为潜在功能预留接口，提前确定好清晰的模块化设计，能有效降低后续需求变更的成本。
- **用户体验与工程实现的平衡**：在力导向图聚类优化、复习时间轴可视化、AI 快捷功能等设计中，体会到功能设计与用户体验之间的权衡，认识到"好用"往往比"实现难度高"更具工程价值。

4.2 反思

- **测试覆盖不足**：当前以手动测试为主，后续可引入 Qt Test 对 `BlockParser`、`InlineParser`、`ReviewStrategy`、`QuestionExtractor` 等模块进行单元测试。
- **AI Key 安全**：API Key 通过 `QSettings` 明文保存，后续需要考虑加密存储。

- **代码可维护性不足**：开发过程中部分模块的命名、注释与职责划分不够清晰，导致后期迭代时理解成本较高。反思认为，可维护性应在编码初期主动保障，可以尝试确定命名规则、划分模块、适当注释、跟进任务清单等方法。

4.3 结语

Qbsidian 作为一款课程大作业作品，已基本实现预期功能。开发过程中，小组成员对 Qt/C++ 桌面应用开发、面向对象设计、数据结构与算法、文件操作等有了更深入的理解，也体会到模块化设计与团队协作在复杂项目中的重要性。